



DOCUMENTAÇÃO DO DESENVOLVEDOR

Sistema de Gerenciamento de Usuários e Pets

Disponível em: <<https://github.com/mlrlima/tela-login-angular>>

Versão 1.0
Julho de 2026

Sumário

Sumário.....	2
1. Visão geral.....	3
2. Arquitetura.....	3
3. Stack tecnológica.....	3
4. Estrutura de pastas.....	4
5. Modelo de dados (diagrama de classes).....	4
6. Camadas do back-end.....	5
7. Autenticação e autorização.....	6
8. Referência da API REST.....	7
9. Front-end Angular.....	8
10. Como rodar.....	8
11. Banco de dados.....	8
12. Limitações conhecidas e próximos passos.....	9

1. Visão geral

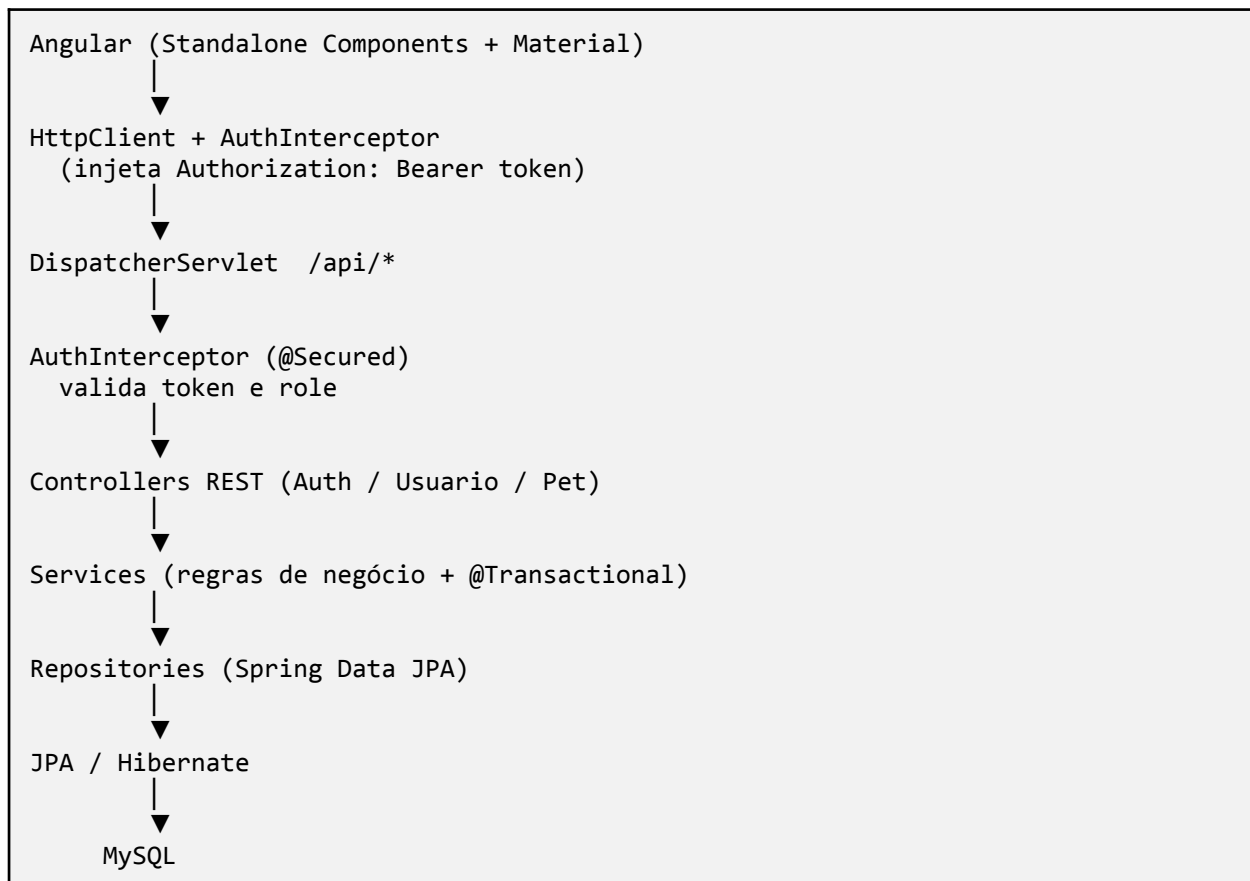
O PetManager é uma aplicação web full-stack:

- Back-end: API REST em Spring Boot, empacotada como WAR e implantada em um Tomcat 11 externo.
- Front-end: Angular (standalone components) com Angular Material, consumindo a API via HttpClient.
- Persistência: MySQL via Spring Data JPA.
- Segurança: autenticação por token opaco (UUID) tipo Bearer, com controle de acesso por role (ADMIN / USER).

Em produção, o Angular é compilado e servido pelo próprio Tomcat, no mesmo contexto da API (/tela-login-angular); um SpaFallbackController redireciona as rotas do Angular para o index.html.

2. Arquitetura

Fluxo de requisição:



Padrão em camadas: **controller** → **service** → **repository** → **model**, com pacotes de apoio **security**, **config** e **util**.

3. Stack tecnológica

Camada	Tecnologia
Linguagem (back)	Java 17+
Framework (back)	Spring Boot (spring-boot-starter-web, -data-jpa, -validation, -tomcat)
Namespace	Jakarta (jakarta.persistence, jakarta.validation)
ORM / dados	Spring Data JPA + Hibernate
Banco	MySQL (mysql-connector-j)
Build (back)	Maven · empacotamento WAR (finalName: tela-login-angular)
Servidor	Tomcat 11 externo
Front-end	Angular (standalone components), Angular Material, TypeScript
Build (front)	Angular CLI (ng serve / ng build)

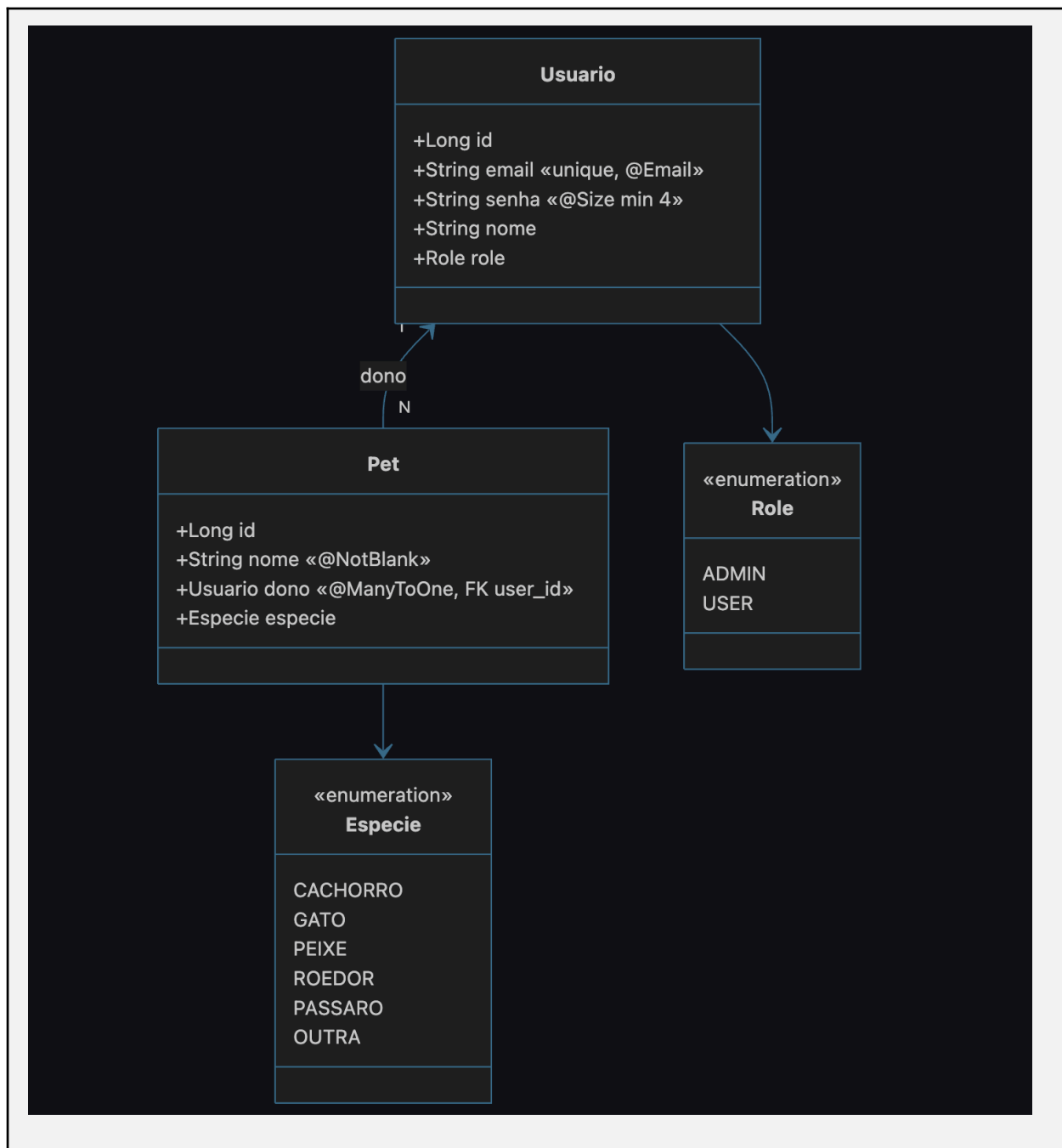
⚠ Versões antigas deste projeto usavam Spring MVC puro + Weld/CDI. Isso foi substituído por Spring Boot — não há mais Weld/CDI. Ao encontrar menções a "Spring MVC" ou "CDI" em textos antigos, considere desatualizado.

4. Estrutura de pastas

```
tela-login-angular/
├── pom.xml                # build do back-end (Spring Boot, WAR)
├── package.json           # script "dev" (roda back + front juntos)
├── src/main/java/
│   ├── config/            # Application (main), GlobalExceptionHandler
│   └── controller/        # AuthController, UsuarioController,
│                           PetController, SpaFallbackController
│   ├── model/             # Usuario, Pet, Role, Especie
│   ├── repository/        # UsuarioRepository, PetRepository(Spring Data)
│   ├── security/          # GeradorToken, AuthInterceptor, Secured
│   └── service/           # UsuarioService, PetService
├── src/main/resources/    # application.properties, etc.
└── angular-frontend/src/app/
    ├── login/ · criar-usuario/    # telas de autenticação
    ├── gestao-pets/ · pet-form/   # CRUD de pets
    ├── gestao-usuarios/ · usuario-form/ # CRUD de usuários
    ├── services/                 # auth.service,pet,usuario (HttpClient)
    ├── guards/                   # auth-guard, admin-guard, login-guard
    ├── interceptors/             # auth.interceptor (Bearer token)
    └── app.routes.ts             # rotas + guards
```

5. Modelo de dados (diagrama de classes)

Diagrama de classes:



Usuario — tabela `usuario`; `email` é único; validações Bean Validation (`@NotBlank`, `@Email`, `@Size`).

Pet — tabela `pet`; relacionamento N:1 com **Usuario** via FK `user_id`; `especie` persistida como `String` (`@Enumerated(EnumType.STRING)`).

`id` gerado com `GenerationType.IDENTITY` (auto-incremento). `equals/hashCode` baseados no `id`.

6. Camadas do back-end

Controllers (`@RestController`): expõem os endpoints e delegam para os services. Ex.: `PetController` tem `@Secured` na classe (todos os endpoints exigem autenticação) e injeta `PetService` via `@Autowired`.

Services (**@Service**): regras de negócio, permissão e transações. Ex.: **PetService**:

```
// ADMIN vê todos; USER vê só os seus
public List<Pet> getAllPets(HttpServletRequest request) {
    Usuario logado = logado(request);
    if (logado.getRole() == Role.ADMIN) return petRepository.findAll();
    return petRepository.findByDono_Id(logado.getId());
}

@Transactional
public Pet createPet(Pet pet, HttpServletRequest request) {
    pet.setDono(logado(request)); // o dono é SEMPRE quem criou
    pet.setId(null);
    return petRepository.save(pet);
}
```

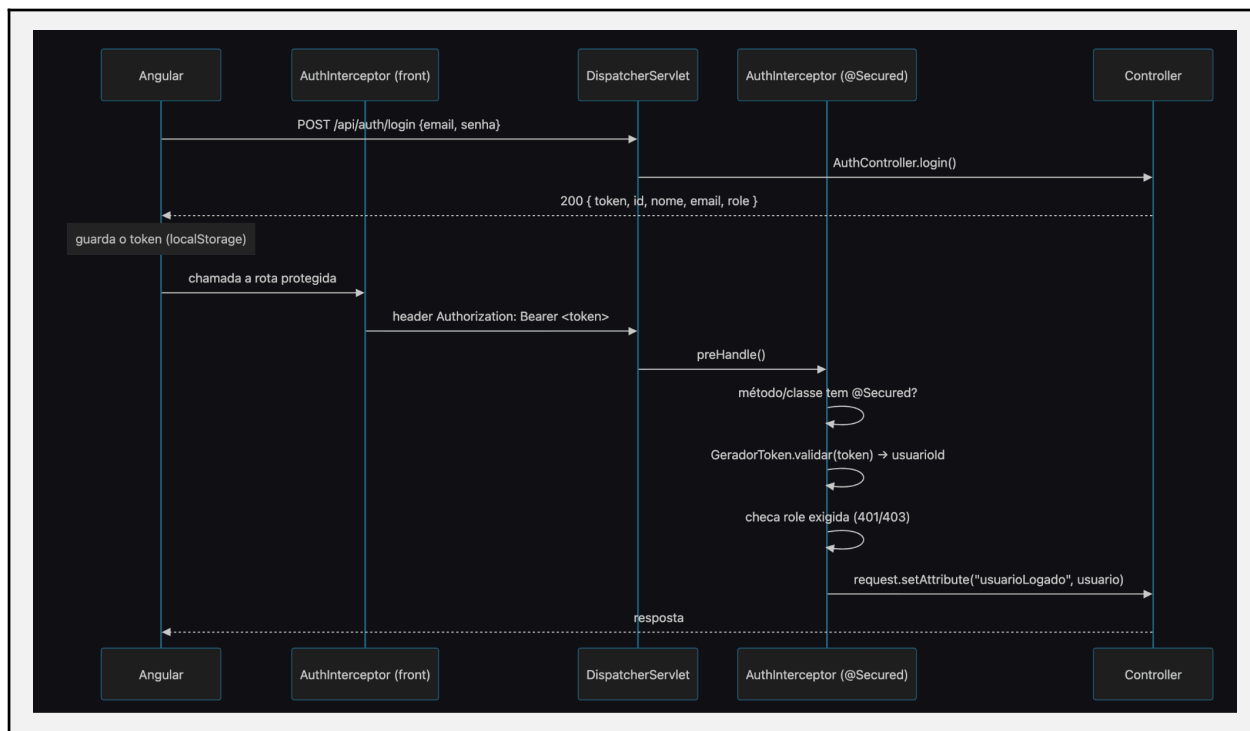
Repositories (Spring Data JPA): interfaces que estendem **JpaRepository<Entidade, Long>**; o Spring gera a implementação e as queries a partir do nome do método:

```
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    Optional<Usuario> findByEmail(String email); // SELECT ... WHERE email = ?
}
public interface PetRepository extends JpaRepository<Pet, Long> {
    List<Pet> findByDono_Id(Long donoId); // pets de um dono
}
```

Config: **Application** (classe de bootstrap Spring Boot) e **GlobalExceptionHandler** (**@RestControllerAdvice**) para tratamento centralizado de erros → respostas HTTP consistentes.

7. Autenticação e autorização

Fluxo de um request protegido (diagrama de sequência):



GeradorToken: gera um UUID no login e guarda **token** → **idUsuário** em memória (**ConcurrentHashMap**); **validar()** e **invalidar()** (logout).

@Secured: anotação própria: **@Secured** = qualquer usuário logado; **@Secured(Role.ADMIN)** = só ADMIN.

AuthInterceptor (HandlerInterceptor): roda antes do controller: exige **Authorization: Bearer**, valida o token, carrega o usuário, confere a role e injeta **usuarioLogado** no request.

No front-end, o **auth.interceptor.ts** injeta o header automaticamente, e os guards (**auth-guard**, **admin-guard**, **login-guard**) replicam as regras de role nas rotas do Angular.

8. Referência da API REST

Prefixo base: **/api**. "Protegido = sim" exige header **Authorization: Bearer <token>**.

Método	Rota	Protegido	Descrição
POST	/api/auth/login	Não	Autentica; retorna token + dados do usuário
POST	/api/auth/logout	Bearer	Invalida o token
GET	/api/auth/me	Sim	Retorna o usuário autenticado
POST	/api/usuario	Não	Cadastro público de usuário (role forçada = USER)
GET	/api/usuario/all	Sim (ADMIN)	Lista todos os usuários
GET	/api/usuario/{id}	Sim	Busca usuário (ADMIN ou o próprio)
PUT	/api/usuario	Sim	Atualiza usuário (ADMIN ou o próprio)
DELETE	/api/usuario/{id}	Sim	Exclui usuário (cascata nos pets)
GET	/api/pet/all	Sim	Lista pets (ADMIN: todos; USER: os seus)

GET	/api/pet/{id}	Sim	Busca pet (ADMIN ou dono)
POST	/api/pet	Sim	Cria pet (dono = usuário logado)
PUT	/api/pet	Sim	Atualiza pet (ADMIN ou dono)
DELETE	/api/pet/{id}	Sim	Exclui pet (ADMIN ou dono)

Confira as rotas exatas em `controller/**` — esta tabela reflete o estado atual do código, mas o código é sempre a referência final.

9. Front-end Angular

Standalone components (sem `NgModule` de app): cada tela é um componente autônomo com seus próprios `imports`.

- Angular Material para UI (cards, tabelas, form fields, selects, ícones).
- Rotas (`app.routes.ts`): `login`, `criar-usuario` (com `loginGuard`), `pets`, `pets/novo`, `pets/:id`, `usuarios`, `usuarios/:id` (com `authGuard`).
- Services (`services/`): `auth.service`, `pet`, `usuario` — encapsulam as chamadas `HttpClient` à API (`environment.apiUrl`).
- Interceptor (`interceptors/auth.interceptor.ts`): adiciona o Bearer token a cada requisição.
- Guards (`guards/`): protegem rotas conforme login e role.
- Ambiente: `environment.ts` define `apiUrl` (ex.: `http://localhost:8080/tela-login-angular`).

10. Como rodar

1. Buildar o Angular: `cd angular-frontend && ng build`.
2. Copiar todos os arquivos gerados em `angular-frontend/dist` para `src/main/resources/static` (observação: os arquivos devem estar diretamente dentro da pasta `static`, não deve haver nenhuma outra subpasta).
3. Gerar o WAR: `mvn clean package` → `target/tela-login-angular.war`.
4. Executar através do Tomcat.

11. Banco de dados

Configuração em `src/main/resources/application.properties`:

- `Spring.datasource.url`: conexão MySQL (ex.: `jdbc:mysql://localhost:3306/...`).
- `Spring.jpa.hibernate.ddl-auto`: geração do schema (ex.: `create` recria o banco a cada start; em produção use `validate/none`).
- `spring.jpa.show-sql` / `format_sql`: log das queries.

⚠ Com `ddl-auto=create`, os dados são apagados a cada reinício. Confirme o valor antes de usar em ambiente com dados reais.

12. Limitações conhecidas e próximos passos

1. **Senha em texto puro.** `UsuarioService.getUsuarioByEmailAndSenha` compara `senha` diretamente (`.equals`). Falta hashing (BCrypt).
2. **Entidade exposta na API.** `GET /auth/me`, `/usuario/all` e `/usuario/{id}` retornam a entidade `Usuario` com o campo `senha` no JSON. Falta DTO de resposta (sem senha). (O login já monta um Map sem a senha — falta aplicar o mesmo cuidado no resto).
3. **Token sem expiração e em memória.** `GeradorToken` guarda tokens num `ConcurrentHashMap` — some no restart do servidor e não expira. Evoluir para JWT com expiração (ou store persistente).
4. Sem testes automatizados cobrindo services/controllers (há apenas `.spec.ts` gerados no front). Introduzir JUnit + testes de integração no back-end.
5. `updatePet` confia no `dono` vindo do payload ao checar permissão — revisar para reforçar o dono a partir do usuário logado, evita498449ndo manipulação.